

How Web Applications Actually Work

Expanded student notes and workbook

DB

What this pack is for

These notes are designed for slow reading, class discussion, and revision before the quiz. Do not rush them. Read one section, explain it out loud, then answer the practice prompts.

By the end of the week, you should be able to:

- Explain the difference between a static site and a dynamic app.
- Trace a user action through frontend, backend, database, and response.
- Describe authentication, sessions, cookies, authorization, and roles.
- Explain deployment pieces: VPS, DNS, ports, reverse proxy, service, and logs.
- Use debugging questions to locate where a web app flow has failed.

How to study: First read for understanding. Then re-read with a pen and mark every word you cannot explain. Finally, complete the mini project without looking at the examples.

1. The Big Idea: A Web App Is a Conversation

A web application is not just files on a screen. It is a conversation between the browser, frontend code, backend routes, database tables, and deployment system.

When a student clicks Sign in, the frontend collects input, the backend checks credentials, the database confirms the user, and the browser receives a session cookie. That is a full app flow.

Layer	Job
Browser	Displays the interface and sends user actions.
Frontend	HTML, CSS, and JavaScript that the user touches.
Backend	Trusted server code that handles rules and APIs.
Database	Persistent memory for users, courses, orders, and progress.
Deployment	Domain, server, proxy, logs, and environment variables.

Key idea: The frontend can guide the user, but the backend must make trusted decisions.

Trace what happens when a student opens the LMS homepage.

2. Static Site vs. Dynamic Web Application

A static site sends mostly the same files to everyone. A dynamic app changes based on who is signed in, what data exists, and what permissions the user has.

Question	Static site	Dynamic web app
Same for every visitor?	Mostly yes.	Often no.
Needs login?	Usually no.	Often yes.
Saves progress?	Not by itself.	Yes, through backend and database.
Example	A brochure page.	The LMS student portal.

Your LMS started as pages and files, but became a real application when it gained login, roles, database-backed progress, teacher resources, checkout approval, and admin oversight.

Which three LMS features prove that it is a dynamic app?

3. Frontend: What the User Touches

The frontend is the part students, teachers, and admins see. It includes layout, buttons, forms, messages, video players, and the JavaScript that calls APIs.

- HTML gives structure: headings, forms, sections, tables.
- CSS controls layout, spacing, color, and mobile behavior.
- JavaScript reacts to clicks, validates simple inputs, calls APIs, and updates the screen.

```
form.addEventListener('submit', function (event) {  
  event.preventDefault();  
  fetch('/api/auth/login', { method: 'POST', body: JSON.stringify(data) });  
});
```

Frontend responsibility: Make the workflow clear. Do not make the frontend the only place where security rules live.

What should the frontend check before sending a checkout form?

4. Backend Routes and APIs

The backend exposes routes. A route is a URL path that the server knows how to handle. APIs let the frontend ask the backend to do work.

Route	Possible job
POST /api/auth/login	Check username and password.
GET /api/progress	Return completed lessons and latest quiz data.
POST /api/progress/quiz	Save a quiz attempt.
POST /api/admin/orders/ approve	Approve course access.

The backend should validate input, check the current user, apply the app rule, then read or write the database.

Why should a student not be able to call an admin route successfully?

5. Request and Response Flow

Most web app behavior follows the same shape: user action, request, backend decision, database work, response, UI update.

```
Student clicks Submit Quiz
-> Frontend gathers answers
-> POST /api/progress/quiz
-> Backend checks signed-in user
-> Backend calculates or accepts score data
-> Database saves result
-> Response returns success
-> Frontend shows completion message
```

Why this matters

When something breaks, you can debug the flow step by step. Did the click happen? Did the request send? Did the backend reject it? Did the database save it? Did the response reach the browser?

Trace the flow for teacher video upload from choose file to student playback.

6. Database Integration

A backend becomes useful when it can save and retrieve real app state. The database is where the app remembers users, roles, resources, orders, progress, and quiz attempts.

App need	Database data
Login	Username, password hash, session/user ID.
Course access	Orders and enrollments.
Student progress	Lesson sections, completed timestamps.
Teacher resources	Lesson slug, title, type, URL, description.
Admin oversight	Users, roles, orders, completion records.

If data exists only in localStorage or only in the browser, the teacher cannot reliably see it. For teacher visibility, it must be submitted to the backend and saved.

What student work must be saved in the database so the teacher can grade it?

7. Login, Sessions, and Cookies

Authentication proves who the user is. A session remembers that proof for later requests. A cookie usually carries the session identity in the browser.

After login succeeds, the backend sends a cookie. Later, when the browser asks for progress or submits a quiz, the cookie travels with the request. The backend uses it to know which user is making the request.

Term	Meaning
Authentication	Checking identity.
Session	A saved signed-in state.
Cookie	Small browser storage sent with matching requests.
Logout	Ending or deleting the session.

Production note: Secure cookies should be used with HTTPS in production so session data is protected in transit.

Why does the app need login before saving student progress?

8. Authorization and Roles

Authorization is the question: what is this signed-in user allowed to do? Roles help the backend answer that question.

Role	Should be allowed
Student	View enrolled courses, submit activities, take quizzes.
Teacher	Add resources, upload videos, review student work.
Admin	Approve orders, manage users, oversee teachers and students.

The same login page can authenticate everyone, but the app should route each person to the correct portal and hide actions they should not use.

Important: Hiding a button is not enough. The backend route must also reject unauthorized requests.

Give one action that must be admin-only and one action that must be teacher-only.

9. Validation and Error Handling

Validation checks that data is complete, expected, and safe before the backend trusts it. Error handling explains what went wrong without exposing secrets.

- Required fields should not be empty.
- URLs should look like valid URLs before becoming resources.
- Uploaded files should be checked for type and size.
- A score should not be larger than the total number of questions.
- Errors should be useful, but not reveal passwords, tokens, or database internals.

```
if not current_user:
    return error('Sign in first', 401)
if current_user.role != 'teacher':
    return error('Teacher access required', 403)
```

What should the backend validate before saving a teacher resource?

10. Deployment: From Local App to Live Domain

Deployment is how code becomes a live service. Your laptop can run a local version, but students need a public domain connected to a server.

Piece	Meaning
VPS	The server where the app runs.
Port	The number the app listens on, such as 8765.
Domain	The public name students type.
DNS	Points the domain to the server IP.
Reverse proxy	Receives web traffic and forwards it to the correct app.
Service	Keeps the app running after terminal closes or server restarts.

digitalbrigdeacademy.com

-> DNS A record

-> VPS public IP

-> reverse proxy on ports 80/443

-> Learning MS app on port 8765

Why is a reverse proxy helpful when several apps run on one VPS?

11. Environment Variables and Secrets

Environment variables let production settings live outside the code. This is where database URLs, ports, cookie settings, and secret values belong.

The same code can run locally with a test database and in production with PostgreSQL because configuration comes from environment variables.

Variable	Example purpose
DATABASE_URL	How the app connects to PostgreSQL.
PORT	Which port the backend listens on.
COOKIE_SECURE	Whether cookies require HTTPS.
COOKIE_NAME	The browser cookie name.

Security habit: Never commit real passwords, database credentials, private keys, or tokens into GitHub.

Why should DATABASE_URL be different from ordinary visible code?

12. Logs, Debugging, and Maintenance

A live app needs maintenance. Logs help you understand requests, errors, restarts, and slow behavior.

When a browser shows Bad Gateway, the question is not only 'is the website broken?' The real checks are: is the app running, is the proxy routing correctly, is the database reachable, and do logs show an exception?

Symptom	What to check
Page takes too long	App process, database connection, server load.
Login fails	Credentials, sessions, database users table.
Video not playing	File path, upload size, browser video support.
Bad Gateway	Reverse proxy target and app service status.

What would you check first if the LMS homepage opens but quiz submission fails?

13. Practice: Trace a Full LMS Feature

A strong developer can trace a feature across layers. This practice turns the whole lesson into one clear mental model.

Choose one feature

- Student signs in.
- Student submits quiz.
- Teacher uploads video.
- Admin approves checkout.
- Student downloads lesson notes.

Trace it across layers

What does the user click?

What frontend code runs?

What backend route is called?

What database tables are read or written?

What response should the user see?

Glossary

Use this vocabulary page when reading the lesson or answering quiz questions. The best test is simple: can you explain each term using the LMS as an example?

Term	Meaning
Frontend	The browser-facing part of the app that users interact with.
Backend	The trusted server-side code that handles rules, APIs, and database access.
API	A structured way for frontend and backend to communicate.
Route	A URL path handled by the backend.
Request	A message sent by the browser or client to the server.
Response	The message the server sends back.
Authentication	Checking who the user is.
Authorization	Checking what the user is allowed to do.
Session	Saved signed-in state.
Cookie	Small browser storage often used to carry session identity.
Reverse proxy	A service that receives web traffic and forwards it to the right app.
Environment variable	Configuration stored outside code.

Choose five terms and explain them using one app you use every week.

Final Mini Project

Trace a complete LMS feature from user action to database save and back to the screen. Your goal is to show the full web app flow.

What to produce

- A step-by-step trace across frontend, backend, database, and response.
- The route or API name you would expect.
- The database tables involved.
- One validation rule and one authorization rule.
- One possible failure and the message the user should see.

Rubric

Area	Strong evidence	Needs work
Frontend	User action and browser behavior are clear.	Only the screen is described.
Backend	Routes and rules are included.	Backend is skipped.
Database	Tables and saved state are named.	No persistent data is described.
Deployment/debugging	Failure and log/debug checks are realistic.	No failure handling is shown.

Planning space